Méthode ExtBridge du package levyBridge

Sommaire

Préparation **1**
Principe . . . 1

I - Une trajectoire **3**
Gamma . . . 3
Wiener . . . 4
Inverse gaussien . . . 5
Variance-gamma . . . 5
Meixner . . . 6
Comparaison des deux méthodes . . . 7

II - Plusieurs trajectoires **8**
Gamma . . . 8
Wiener . . . 9
Inverse gaussien . . . 10
Variance-gamma . . . 11
Meixner . . . 12

III - Nombre de décimales et troncature **14**

IV - Plusieurs trajectoires : pas commun (`common_step`) **16**
Accroissements très petits . . . 16

Préparation

Installation des packages depuis GitHub : `levyBridge` pour les ponts et `DegradSim` pour simuler des trajectoires.

```
remotes::install_github("rytracetito/levy_packages", subdir = "levyBridge")
remotes::install_github("rytracetito/levy_packages", subdir = "DegradSim")
```

Principe

Une trajectoire est observée aux instants $0 = t_0 < t_1 < \cdots < t_N$. Les deux méthodes de `levyBridge` tirent, sur une grille régulière, des valeurs qui suivent exactement la loi du processus sachant les observations ; elles ne diffèrent que par la grille.

- **Méthode `"Bridge"`** (Grall-Maës, 2012) : grille $t'_k = k\,t_N/N$, avec autant de points que d'observations. Les instants d'inspection ne sont en général pas sur la grille.
- **Méthode `"ExtBridge"`** : les instants sont des multiples de 10^{-d}, où d est le nombre de décimales, et le pas est le plus grand tel que tous les instants d'inspection soient sur la grille :

$$h = 10^{-d}\,\mathrm{pgcd}\big(10^d t_1, \ldots, 10^d t_N\big), \qquad t'_k = k\,h, \quad k = 0, \ldots, t_N/h.$$

Le pseudo-échantillon contient donc toutes les observations. La grille a $t_N/h \geq N$ pas, avec égalité si et seulement si les inspections sont régulières : la méthode renvoie alors les observations elles-mêmes.

Dans les deux cas, la grille est parcourue de gauche à droite : un point de grille égal à un instant d'inspection garde la valeur observée, les autres sont tirés dans le pont entre la dernière valeur connue à gauche et l'inspection suivante. Sous le modèle, les accroissements reconstruits sont indépendants, de la loi de X_h.

Le nombre de décimales d est trouvé automatiquement (argument `digits = NULL`, partie III).

Paramètres utilisés :

Processus	Pont	Rééchantillonnage	`theta`	Utilisés par le pont
Gamma	`gamma_bridge`	`gamma_resampling`	`a = 2`, `b = 3`	`a`
Wiener	`wiener_bridge`	`wiener_resampling`	`mu = 0.5`, `sigma = 1.5`	`sigma`
Inverse gaussien	`ig_bridge`	`ig_resampling`	`mu = 1`, `eta = 3`	`eta`
Variance-gamma	`vg_bridge`	`vg_resampling`	`theta = 0.5`, `sigma = 1.5`, `nu = 2`	`theta`, `sigma`, `nu`
Meixner	`meixner_bridge`	`meixner_resampling`	`a = 1`, `b = 0.3`, `d = 1.5`, `m = 0.2`	`a`, `d`, `m`

I - Une trajectoire

Une trajectoire observée à 11 instants irréguliers entre 0 et 22.5, donnés avec une décimale. Les écarts entre inspections, en dixièmes, ont pour pgcd 1 : le pas de la grille `"ExtBridge"` est $h = 0.1$, soit 225 pas, contre 10 pas de 2.25 pour la méthode `"Bridge"`.

```
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
```

Pour chaque processus, on simule la trajectoire, on applique la méthode `"ExtBridge"`, puis on vérifie que les valeurs observées sont retrouvées exactement sur la grille.

Gamma

```
set.seed(1)
x <- sim_gamma(t, a = 2, b = 3)
r <- gamma_resampling(x, theta = c(a = 2, b = 3), method = "ExtBridge")
r
```

```
## 1 trajectory(ies) resampled on a regular grid by the gamma bridge (method "ExtBridge")
## parameters: a = 2, b = 3
## time step teq: 0.1
##
##   id   t            X
## 1  1 0.0 0.000000e+00
## 2  1 0.1 1.393433e-05
## 3  1 0.2 2.866446e-03
## 4  1 0.3 2.866447e-03
## 5  1 0.4 4.715224e-03
## 6  1 0.5 1.670398e-01
```

```
attr(r, "teq")
```

```
## [1] 0.1
```

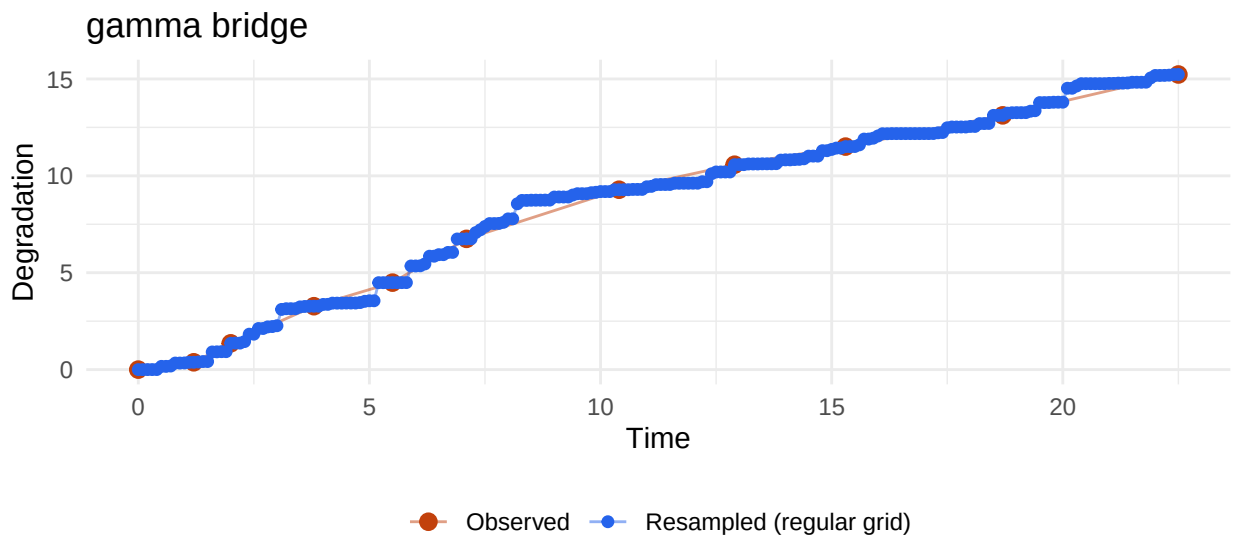
```
nrow(r[[1]])
```

```
## [1] 226
```

```
all.equal(r[[1]]$X[match(t, r[[1]]$t)], x[, "X1"])
```

```
## [1] TRUE
```

```
plot_bridge(r)
```



Wiener

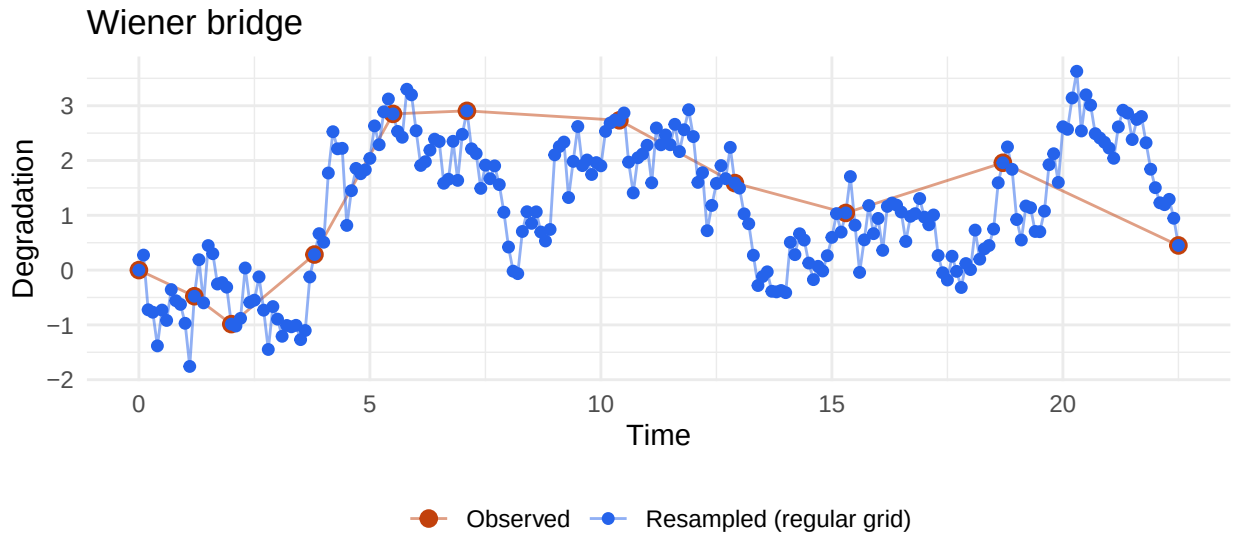
```
x <- sim_wiener(t, mu = 0.5, sigma = 1.5)
r <- wiener_resampling(x, theta = c(mu = 0.5, sigma = 1.5), method = "ExtBridge")
attr(r, "teq")
```

```
## [1] 0.1
```

```
all.equal(r[[1]]$X[match(t, r[[1]]$t)], x[, "X1"])
```

```
## [1] TRUE
```

```
plot_bridge(r)
```



Inverse gaussien

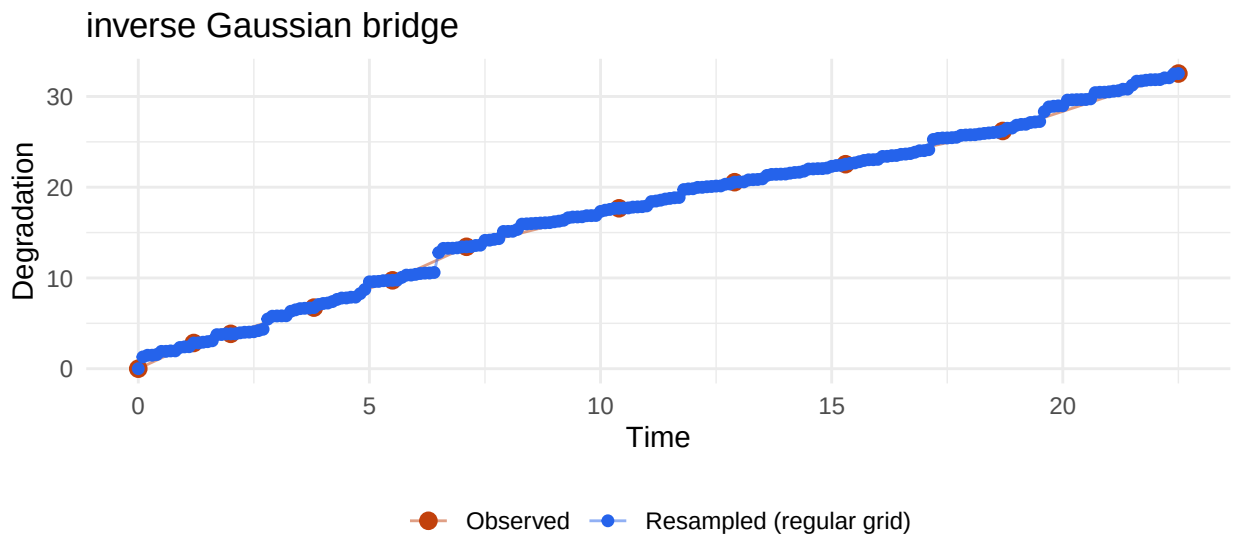
```
x <- sim_ig(t, mu = 1, eta = 3)
r <- ig_resampling(x, theta = c(mu = 1, eta = 3), method = "ExtBridge")
attr(r, "teq")
```

```
## [1] 0.1
```

```
all.equal(r[[1]]$X[match(t, r[[1]]$t)], x[, "X1"])
```

```
## [1] TRUE
```

```
plot_bridge(r)
```



Variance-gamma

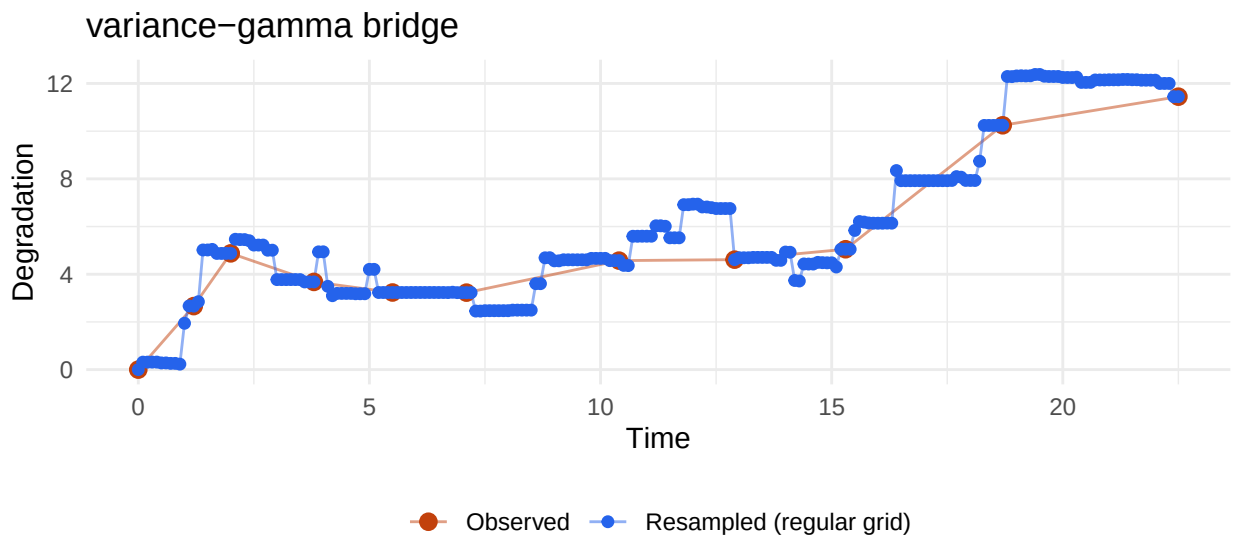
```
x <- sim_vg(t, theta = 0.5, sigma = 1.5, nu = 2)
r <- vg_resampling(x, theta = c(theta = 0.5, sigma = 1.5, nu = 2), method = "ExtBridge")
attr(r, "teq")
```

```
## [1] 0.1
```

```
all.equal(r[[1]]$X[match(t, r[[1]]$t)], x[, "X1"])
```

```
## [1] TRUE
```

```
plot_bridge(r)
```



Meixner

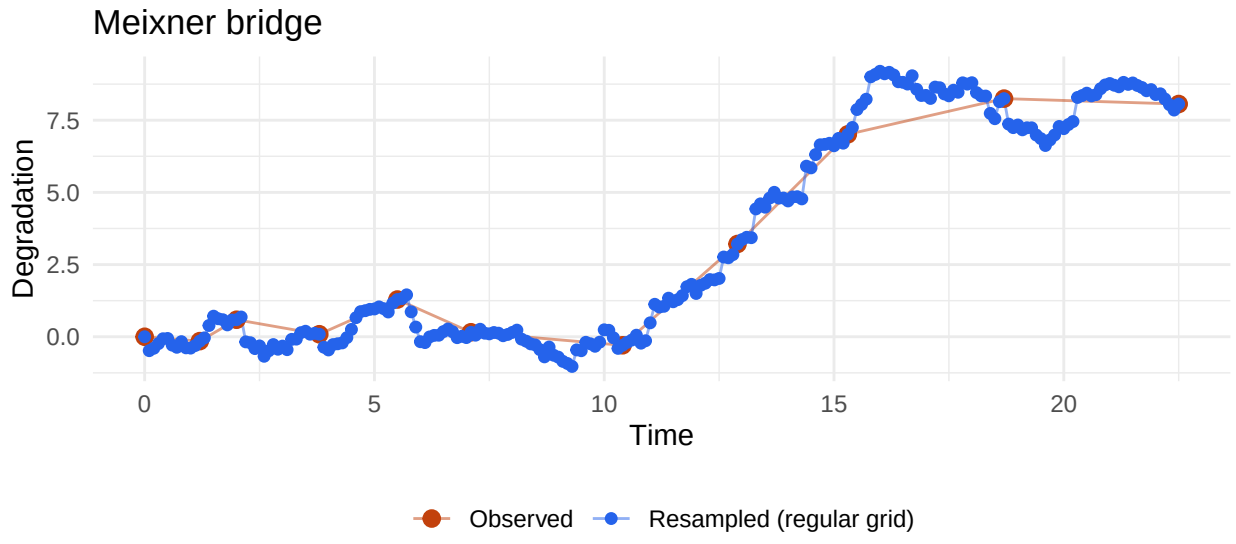
```
x <- sim_meixner(t, a = 1, b = 0.3, d = 1.5, m = 0.2)
r <- meixner_resampling(x, theta = c(a = 1, b = 0.3, d = 1.5, m = 0.2), method = "ExtBridge")
attr(r, "teq")
```

```
## [1] 0.1
```

```
all.equal(r[[1]]$X[match(t, r[[1]]$t)], x[, "X1"])
```

```
## [1] TRUE
```

```
plot_bridge(r)
```



Comparaison des deux méthodes

Sur la trajectoire gamma, les deux méthodes donnent des grilles différentes : 10 pas de 2.25 pour `"Bridge"`, grille qui ne passe ici que par les inspections 0 et 22.5 ; 225 pas de 0.1 pour `"ExtBridge"`, grille qui passe par toutes les inspections.

```
set.seed(1)
x <- sim_gamma(t, a = 2, b = 3)
r_b <- gamma_resampling(x, theta = c(a = 2, b = 3), method = "Bridge")
r_e <- gamma_resampling(x, theta = c(a = 2, b = 3), method = "ExtBridge")
data.frame(methode = c("Bridge", "ExtBridge"),
           pas = c(attr(r_b, "teq"), attr(r_e, "teq")),
           points = c(nrow(r_b[[1]]), nrow(r_e[[1]])),
           inspections_sur_la_grille = c(sum(t %in% r_b[[1]]$t), sum(t %in% r_e[[1]]$t)))
```

```
##     methode  pas points inspections_sur_la_grille
## 1    Bridge 2.25     11                         2
## 2 ExtBridge 0.10    226                        11
```

Avec des inspections régulières, les deux méthodes renvoient les observations elles-mêmes :

```
t_reg <- seq(0, 20, by = 2)
x_reg <- sim_gamma(t_reg, a = 2, b = 3)
r_reg <- gamma_resampling(x_reg, theta = c(a = 2, b = 3), method = "ExtBridge")
all.equal(r_reg[[1]]$X, x_reg[, "X1"])
```

```
## [1] TRUE
```

```
all.equal(gamma_resampling(x_reg, theta = c(a = 2, b = 3), method = "Bridge")[[1]]$X, x_reg[, "X1"])
```

```
## [1] TRUE
```

```
attr(r_reg, "teq")
```

```
## [1] 2
```

II - Plusieurs trajectoires

20 trajectoires inspectées entre 0 et 40, de plus en plus souvent, chacune avec ses propres instants (de 11 à 30 instants), arrondis à une décimale. La méthode `"ExtBridge"` construit une grille et un pas pour chaque trajectoire.

```
plan <- inspection_times("decreasing", n = 10:29, n_traj = 20, tmax = 40, digits = 1)
```

Gamma

```
set.seed(2)
x <- sim_gamma(plan, a = 2, b = 3)
r <- gamma_resampling(x, theta = c(a = 2, b = 3), method = "ExtBridge")
r
```

```
## 20 trajectory(ies) resampled on a regular grid by the gamma bridge (method "ExtBridge")
## parameters: a = 2, b = 3
## time step teq: 0.1
##
##   id   t         X
## 1  1 0.0 0.0000000
## 2  1 0.1 0.5009036
## 3  1 0.2 0.5036295
## 4  1 0.3 0.5069556
## 5  1 0.4 0.5238951
## 6  1 0.5 0.5240004
```

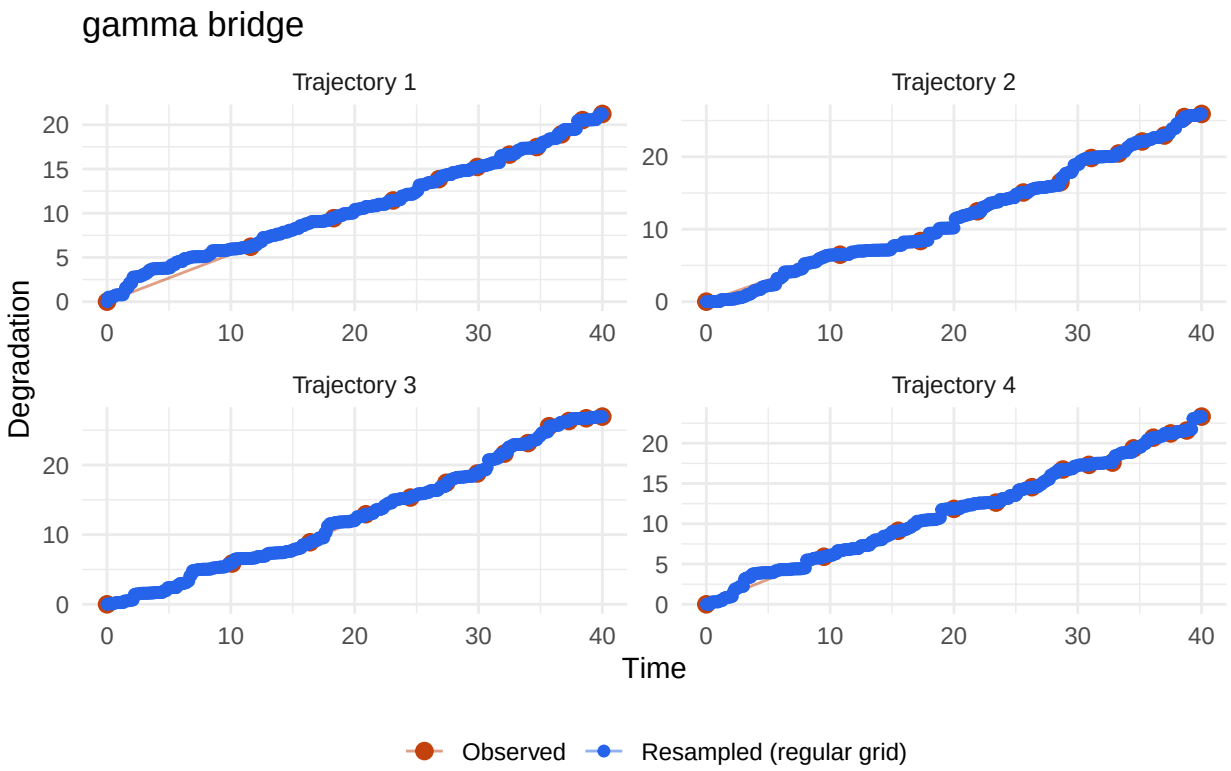
```
summary(attr(r, "teq"))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     0.1     0.1     0.1     0.1     0.1     0.1
```

```
head(attr(r, "dx")[[1]])
```

```
## [1] 0.5009035516 0.0027259156 0.0033260989 0.0169395731 0.0001052243
## [6] 0.1528759247
```

```
plot_bridge(r, trajectory = 1:4)
```

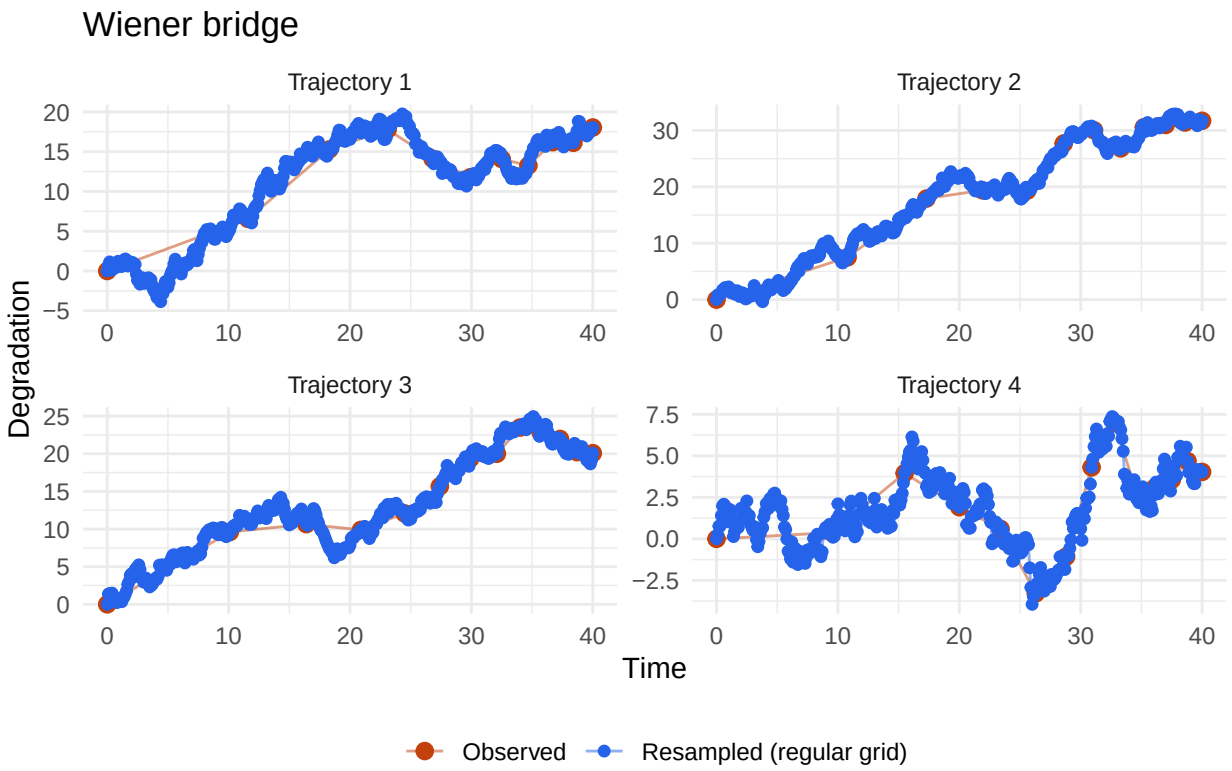


Wiener

```
r <- wiener_resampling(sim_wiener(plan, mu = 0.5, sigma = 1.5), theta = c(mu = 0.5, sigma = 1.5), metho
summary(attr(r, "teq"))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     0.1     0.1     0.1     0.1     0.1     0.1
```

```
plot_bridge(r, trajectory = 1:4)
```

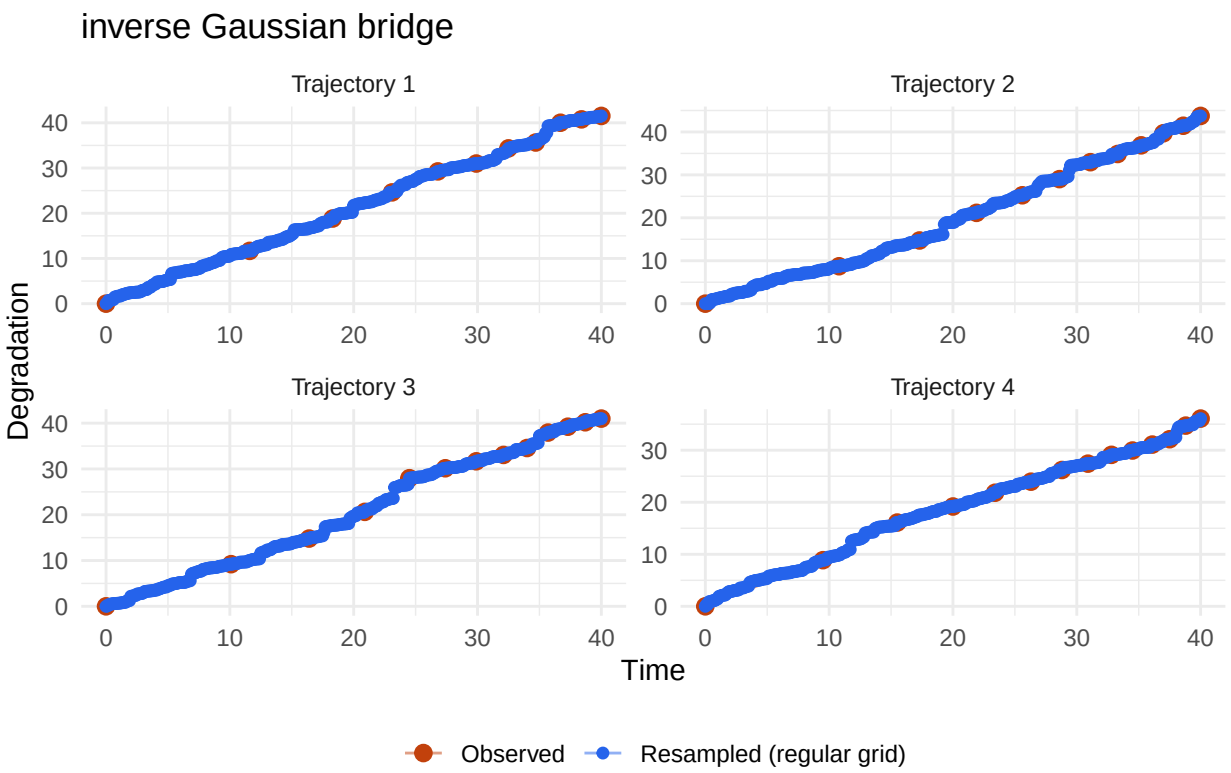


Inverse gaussien

```
r <- ig_resampling(sim_ig(plan, mu = 1, eta = 3), theta = c(mu = 1, eta = 3), method = "ExtBridge")
summary(attr(r, "teq"))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     0.1     0.1     0.1     0.1     0.1     0.1
```

```
plot_bridge(r, trajectory = 1:4)
```

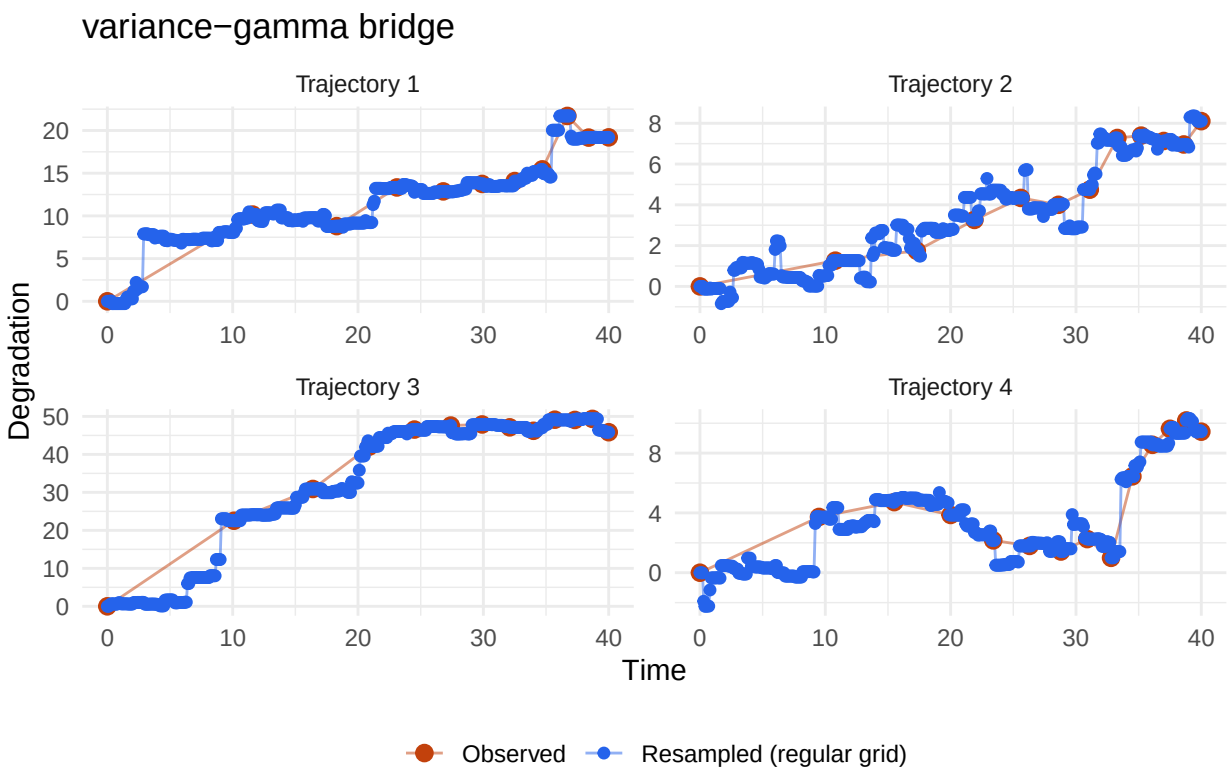


Variance-gamma

```
x <- sim_vg(plan, theta = 0.5, sigma = 1.5, nu = 2)
r <- vg_resampling(x, theta = c(theta = 0.5, sigma = 1.5, nu = 2), method = "ExtBridge")
summary(attr(r, "teq"))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     0.1     0.1     0.1     0.1     0.1     0.1
```

```
plot_bridge(r, trajectory = 1:4)
```

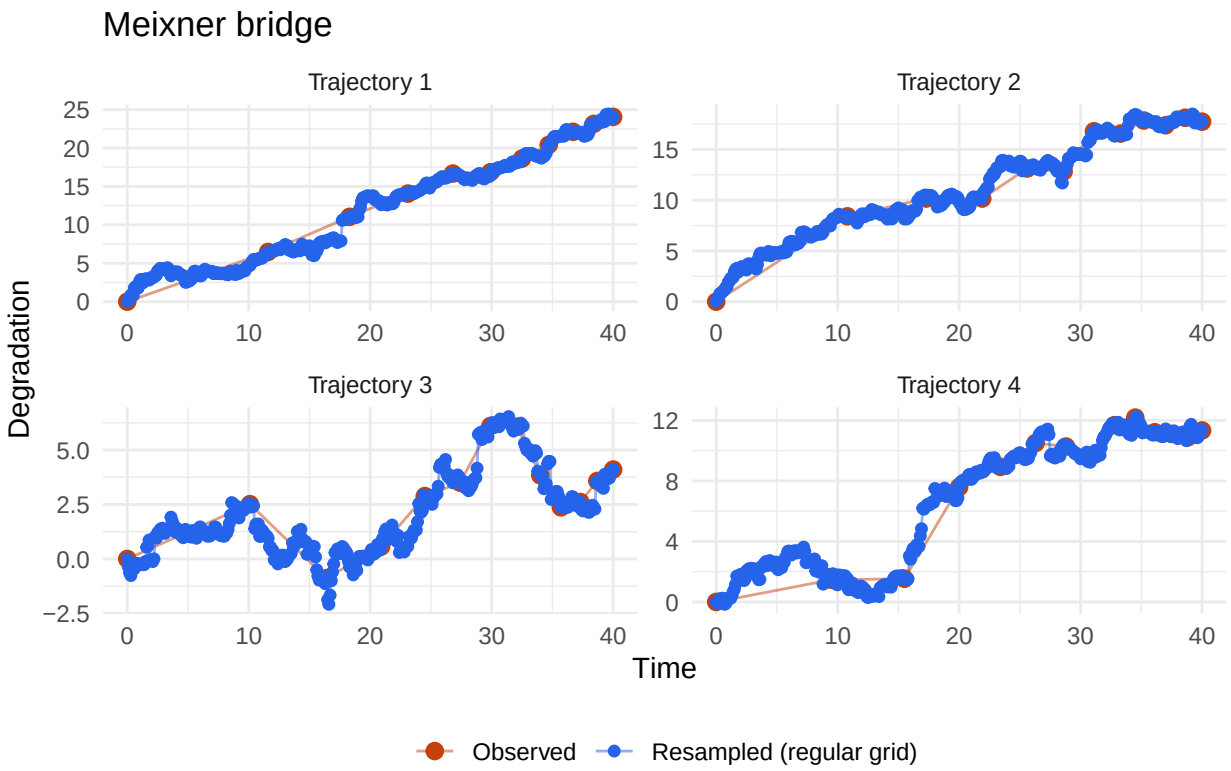


Meixner

```
x <- sim_meixner(plan, a = 1, b = 0.3, d = 1.5, m = 0.2)
r <- meixner_resampling(x, theta = c(a = 1, b = 0.3, d = 1.5, m = 0.2), method = "ExtBridge")
summary(attr(r, "teq"))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     0.1     0.1     0.1     0.1     0.1     0.1
```

```
plot_bridge(r, trajectory = 1:4)
```


Meixner bridge
Trajectory 1
Trajectory 2
Trajectory 3
Trajectory 4
Degradation
Time
Observed
Resampled (regular grid)

III - Nombre de décimales et troncature

Avec `digits = NULL` (valeur par défaut), le nombre de décimales d est le plus petit entier, cherché entre 0 et 6, tel que tous les instants soient des multiples de 10^{-d}. Ici les instants ont deux décimales et le pgcd des écarts, en centièmes, vaut 5 : le pas est 0.05.

```
x2 <- data.frame(t = c(0, 0.35, 1.2, 1.85, 2.5), X = c(0, 0.3, 1.1, 1.4, 2.2))
attr(gamma_resampling(x2, theta = c(a = 2, b = 3), method = "ExtBridge"), "teq")
```

```
## [1] 0.05
```

L'argument `digits` impose le nombre de décimales : les instants sont alors tronqués (partie entière inférieure de $10^d t$), avec un avertissement, et les instants tronqués sont ceux de la grille.

```
r <- gamma_resampling(x2, theta = c(a = 2, b = 3), method = "ExtBridge", digits = 1)
```

```
## Warning: the inspection times are truncated to 1 decimal place
```

```
attr(r, "teq")
```

```
## [1] 0.1
```

```
attr(r, "observed")[[1]]
```

```
##     t   X
## 1 0.0 0.0
## 2 0.3 0.3
## 3 1.2 1.1
## 4 1.8 1.4
## 5 2.5 2.2
```

Des instants tirés au hasard ne sont des multiples d'aucune puissance 10^{-d} avec $d \leq 6$: la fonction s'arrête et demande `digits`. Avec `digits = 2`, les instants sont tronqués à deux décimales, avec un avertissement.

```
set.seed(3)
t3 <- c(0, sort(runif(6, 0, 10)))
x3 <- sim_wiener(t3, mu = 0.5, sigma = 1.5)
try(wiener_resampling(x3, theta = c(mu = 0.5, sigma = 1.5), method = "ExtBridge"))
```

```
## Error : the inspection times have more than 6 decimal places: give the number of decimal places with
```

```
r <- wiener_resampling(x3, theta = c(mu = 0.5, sigma = 1.5), method = "ExtBridge", digits = 2)
```

```
## Warning: the inspection times are truncated to 2 decimal places
```

```
round(t3, 4)
```

```
## [1] 0.0000 1.6804 3.2773 3.8494 6.0210 6.0439 8.0752
```

```
attr(r, "observed")[[1]]$t
```

```
## [1] 0.00 1.68 3.27 3.84 6.02 6.04 8.07
```

```
attr(r, "teq")
```

```
## [1] 0.01
```

Deux instants confondus après la troncature donnent une erreur : il faut alors augmenter `digits`.

```
try(gamma_resampling(data.frame(t = c(0, 1.01, 1.04), X = c(0, 0.5, 0.9)), theta = c(a = 2, b = 3), meth
```

```
## Error in ext_grids(list(t), digits, TRUE) :
##   two inspection times are equal after truncation to 1 decimal place: increase digits
```

Enfin, une grille de plus de 10^5 pas donne un avertissement, car le rééchantillonnage devient long : par exemple des instants à trois décimales jusqu'à 400.

```
r <- gamma_resampling(data.frame(t = c(0, 1.234, 400), X = c(0, 0.4, 150)), theta = c(a = 2, b = 3), me
```

```
## Warning: the grid has 2e+05 time steps: the resampling can be slow
```

```
nrow(r[[1]])
```

```
## [1] 200001
```

IV - Plusieurs trajectoires : pas commun (`common_step`)

Sans autre précision, chaque trajectoire reçoit son propre pas h (le pgcd de ses propres écarts). Deux trajectoires de pas différents donnent des accroissements de lois différentes (X_{h_1} et X_{h_2}) : chacune est un échantillon i.i.d., mais les deux pas sont différents.

```
liste <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.8, 2.1, 4.3)),
              data.frame(t = c(0, 0.5, 1.25, 2), X = c(0, 0.4, 1.1, 1.6)))
r_propre <- gamma_resampling(liste, theta = c(a = 2, b = 3), method = "ExtBridge")
attr(r_propre, "teq")
```

```
## [1] 1.00 0.25
```

```
sapply(r_propre, nrow) - 1
```

```
## [1] 6 8
```

Avec `common_step = TRUE`, h est le pgcd de tous les écarts de toutes les trajectoires, ici 0.25 : toutes les inspections restent sur leur grille, et tous les accroissements reconstruits ont la loi de $X_{0.25}$. Toutes les trajectoires ont le même pas.

```
r_commun <- gamma_resampling(liste, theta = c(a = 2, b = 3), method = "ExtBridge", common_step = TRUE)
attr(r_commun, "teq")
```

```
## [1] 0.25 0.25
```

```
sapply(r_commun, nrow) - 1
```

```
## [1] 24  8
```

```
all(c(0, 1, 3, 6) %in% r_commun[[1]]$t)
```

```
## [1] TRUE
```

```
head(attr(r_commun, "dx")[[1]])
```

```
## [1] 0.2109815392 0.2550334998 0.0008944322 0.3330905288 0.0048228312
## [6] 0.1119873177
```

Avec les 20 trajectoires de la partie II (instants à une décimale), le pas commun vaut 0.1, comme le pas propre de chaque trajectoire :

```
set.seed(2)
r_all <- gamma_resampling(sim_gamma(plan, a = 2, b = 3), theta = c(a = 2, b = 3), method = "ExtBridge",
unique(attr(r_all, "teq"))
```

```
## [1] 0.1
```

Accroissements très petits

Les accroissements reconstruits sont calculés directement par le pont, et non par différence de niveaux : un accroissement très petit garde toute sa précision et n'est pas arrondi à 0, même pour une petite forme ah (ici $ah = 0.05$). Seul un accroissement inférieur au plus petit nombre représentable (environ 10^{-308}), ce qui demande des formes extrêmement petites, vaut 0.

```
x_small <- sim_gamma(t, a = 0.5, b = 3)
min(unlist(attr(gamma_resampling(x_small, theta = c(a = 0.5, b = 3), method = "ExtBridge"), "dx")))
```

```
## [1] 5.698365e-42
```